8강. 클래스

- 클래스 선언
- 객체 생성
- 클래스 구성 멤버
- 필드 선언 및 사용
- 생성자 선언 및 호출
- 메서드 선언 및 호출
- 정적 멤버
- final 필드와 상수

8-1. 클래스 선언

- 클래스 선언은 객체 생성용 설계도를 작성하는 것이다.
  - 어떻게 객체를 생성(생성자)할 것인가?
  - 객체가 가져야 할 데이터(필드)는 무엇일까 ?
  - 객체가 할 수 있는 동작(메서드)은 무엇이 있을까?
  - 생성자, 필드, 메서드 등 내용을 정의 하는 것
- 클래스 명은 소스 파일명과 동일하게 작성

```
// 클래스 선언
public class 클래스명 {
}
```

```
public class Car {

}
```

- 클래스명 명명 규칙
  - 첫 문자는 대문자, 캐멀 스타일로 작성
  - 첫 문자 숫자 불가, 이후 문자는 숫자와 특수문자($, _) 사용 가능
- 클래스 두가지 용도
  - 라이브러리 클래스: 실행할 수 없으며, 다른 클래스에서 이용하는 클래스
  - 실행 클래스 : main() 메서드를 가지고 있는 실행 가능한 클래스

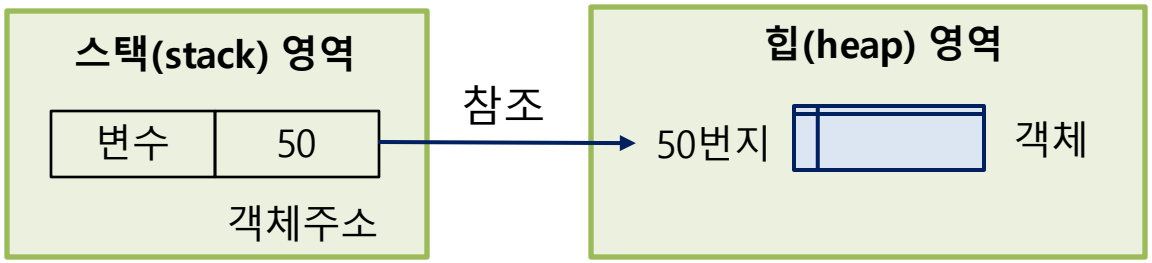
8-2. 객체 생성

➢ 정의한 클래스로 객체 생성 연산자인 new 로 객체(인스턴스) 생성

클래스 변수 = new 클래스();
생성자

```
Car car = new Car();
```

➢ 클래스 변수의 객체 참조 메모리 구조




```
public class CarEx {
    public static void main(String[] args) {
        Car car1 = new Car();
        System.out.println("car1 변수가 Car 객체를 참조합니다.");

        Car car2 = new Car();
        System.out.println("car2 변수가 Car 객체를 참조합니다.");
    }
}
```

8-3. 클래스 구성멤버

➢ 클래스 구성 멤버는 생성자, 필드, 메서드가 있다.

```
public class Car {
    // 필드 선언
    String name;

    // 생성자 선언
    Car() {
    }

    // 메서드 선언
    void run() {
    }
}
```

필드 : 객체의 데이터 저장

생성자 : 객체 생성 시 초기화처리

메서드 : 객체의 동작처리

- 필드 : 객체의 데이터를 저장하며, 선언 형태는 변수와 유사하나 쓰임새는 다르다.
- 생성자 : 객체 생성 시 초기화 처리를 하며 선언 형태는 메서드와 유사하나 리턴 타입이 없고 이름은 클래스 명과 동일
- 메서드 : 객체가 수행할 동작으로 객체 내부에서 메서드로 부른다. 객체와 객체사이의 상호 작용을 담당한다.

8-4. 필드 선언 및 사용

- 필드를 선언하는 방법은 변수를 선언하는 방법과 동일하며, 반드시 클래스 블록에서 선언되어야 한다.

타입 필드명 [= 초기값] ;

```
public class Car {
    String model;
    int speed = 100;
    Tire tire = new Tire();
}
```

- 필드 타입 별 기본값(초기 값 미제공시 객체 생성 시 자동으로 기본값으로 초기화)
  - 기본 타입
    - 정수형 : byte[], short[], int[] -> 0, long[] -> 0L
    - 실수형 : double[] -> 0.0, float[] -> 0.0F
    - 논리형 : boolean[] -> false
    - 문자형 : char[] -> ₩0000 (null character)
  - 참조 타입
    - 클래스[], 인터페이스[] : null

8-4. 필드 선언 및 사용

- 필드를 사용하는 것은 필드 값을 읽고 변경하는 것을 말한다. 단, 객체 생성 후 가능하다.
- 필드는 객체 내부의 생성자와 메서드에서 사용 가능하고, 객체 외부에서도 접근해서 사용할 수 있다.

```
public class Car {
    // 필드 선언
    String company = "기아자동차";
    String model = "쏘렌토";
    String color = "검정";
    int maxSpeed = 220;
    int speed;
}
```

8-4. 필드 선언 및 사용

```
public class CarEx {
    public static void main(String[] args) {
        // Car 객체 생성
        Car myCar = new Car();

        // Car 객체 필드값 읽기
        System.out.println("제조사 : " + myCar.company);
        System.out.println("모델명 : " + myCar.model);
        System.out.println("색상 : " + myCar.color);
        System.out.println("최고속도 : " + myCar.maxSpeed);
        System.out.println("현재속도 : " + myCar.speed);

        // Car 객체의 필드값 변경
        myCar.speed = 50;
        System.out.println("수정된 현재속도 : " + myCar.speed);
    }
}
```

8-4. 생성자 선언 및 호출

- 생성자(Constructor)는 객체를 초기화하는 특별한 종류의 메서드이다.
  - 역할
    - 생성자의 기본 역할은 객체의 초기화이다. 객체가 생성될 때 호출되며, 객체의 상태를 초기화(필드 초기화, 메서드 호출 등)하고 객체를 사용할 준비를 하는 것
  - 이름
    - 생성자의 이름은 클래스 명과 동일, 생성자는 메서드이지만 반환 타입을 지정하지 않는다.

```
클래스 변수 = new 클래스();
                  생성자
```

  - 호출
    - 생성자는 new 연산자와 함께 객체를 생성할 때 자동 호출된다.
    - 일반적으로 클래스 내부에 선언되고, 객체가 생성될 때 한번만 호출된다.

8-4. 생성자 선언 및 호출

➢ 기본 생성자(Default Constructor)

- 모든 클래스는 생성자가 존재하며, 하나 이상 가질 수 있다.
- 클래스에 생성자를 명시적으로 정의하지 않으면 컴파일러는 기본 생성자를 자동으로 생성한다.
- 기본 생성자는 매개 변수가 없고 아무 동작도 수행하지 않는다.

```
[public] 클래스() { }
```

← 기본 생성자 선언 형식

```
public class Car {

}
```

컴파일 →

```
public class Car {
    public Car() { }  // 자동 추가
}
```

```
Car myCar = new Car();
```

기본 생성자 호출

- 기본 생성자가 추가되어 new 연산자 뒤에 기본 생성자를 호출할 수 있다.

8-4. 생성자 선언 및 호출

➢ 생성자 직접 선언

- 객체를 다양하게 초기화하기 위해 생성자를 직접 선언할 수 있다.

```
클래스(매개변수, ... ) {
  // 객체의 초기화 코드
}
```

생성자 블록

- 매개변수를 갖는 생성자 선언

```
Car ( String company, String model, String color ) {
    this.company = company;
    this.model = model;
    this.color = color;
}
```

❖ 현재 객체

8-4. 생성자 선언 및 호출

➢ 필드 초기화

- 객체마다 동일한 값을 갖는 다면 필드 선언 시 초기값을 대입하고, 객체마다 다른 값을 가져야 한다면 생성자에서 초기화하는 것이 좋다.

```
public class KiaCar {
    // 필드 선언
    String company = "기아자동차"; // 선언 시 초기화
    String model;
    String color;
    int maxSpeed;
    int speed;

    KiaCar ( String model, String color, int maxSpeed ) {
        this.model = model;          // 생성자에서 초기화처리
        this.color = color;          // 생성자에서 초기화처리
        this.maxSpeed = maxSpeed;    // 생성자에서 초기화처리
    }
}
```

8-4. 생성자 선언 및 호출

➢ **new 연산자로 매개변수를 넘겨 줌**

- new 연산자로 객체 생성 시 생성자를 호출할 때 매개 변수로 필드 값을 전달

```
public class CarEx {
    public static void main(String[] args) {
        // KiaCar 객체 생성 시 매개변수 성성자 호출
        KiaCar myCar = new KiaCar("쏘렌토", "검정", 230);

        // Car 객체 필드값 읽기
        System.out.println("제조사 : " + myCar.company);
        System.out.println("모델명 : " + myCar.model);
        System.out.println("색상 : " + myCar.color);
        System.out.println("최고속도 : " + myCar.maxSpeed);
        System.out.println("현재속도 : " + myCar.speed);
    }
}
```

8-4. 생성자 선언 및 호출

➢ 생성자 오버로딩

- 매개변수를 달리하는 생성자를 여러 개 선언하는 것이다.
- 매개변수의 타입과 수가 달라야 한다. (이름만 변경 시 에러 발생)

```
public class KiaCar {
    // 필드 선언
    String company = "기아자동차";
    String model;
    String color;
    int maxSpeed;
    int speed;

    // 기본 생성자
    KiaCar () {
        this.speed = 50;
    }

    // 생성자 오버로드
    KiaCar ( String model ) {
        this.model = model;
        this.speed = 50;
    }
}
```

8-4. 생성자 선언 및 호출

➢ **생성자 오버로드한 클래스 사용**

```
public class CarEx {
    public static void main(String[] args) {
        // 매개변수 성성자 호출
        KiaCar myCar1 = new KiaCar("쏘렌토");
        System.out.println("모델명 : " + myCar1.model);

        // 기본 생성자 호출
        KiaCar myCar2 = new KiaCar();
        System.out.println("모델명 : " + myCar2.model);
    }
}
```

8-4. 생성자 선언 및 호출

➢ **생성자에서 다른 생성자 호출**

```
public class KiaCar {
    // 필드 선언
    String company = "기아자동차";
    String model;
    String color;
    int maxSpeed;
    int speed;

    // 기본 생성자
    KiaCar () {
        this("쏘렌토");// 매개변수(String model) 가진 생성자 호출
    }

    // 생성자 오버로드
    KiaCar ( String model ) {
        this.model = model;
        this.speed = 50;
    }
}
```

8-5. 메서드 선언 및 호출

- 메서드 선언은 객체의 동작을 실행 블록으로 정의하는 것이다.
- 메서드 호출은 실행 블록을 실제로 실행하는 것이다.

- **메서드 선언**

```
반환타입 메서드명 (매개변수, ... ) {
    // 실행코드 작성
}
```

실행 블록

- 반환 타입(Return Type) : 메서드가 반환하는 값의 데이터 타입을 나타낸다. 반환 값이 없을 경우 void 를 사용한다.
- 메서드 명(Method Name) : 메서드의 고유한 이름을 나타내며, 첫 글자는 소문자로 시작하고, 캐멀 스타일로 작성한다.
- 매개변수(Parameter) : 메서드에 전달되는 값들을 나타낸다. 매개 변수가 없는 경우 괄호 안을 비워 둔다.
- 실행 블록(Method Body) : 메서드가 수행하는 작업을 정의하는 코드 블록이다.

8-5. 메서드 선언 및 호출

➢ **다양한 메서드 선언**

```
public class Calculator {
    // 반환타입: x, 매개변수: X
    void powerOn() {
        System.out.println("전원을 켭니다.");
    }

    // 반환타입: x, 매개변수: O
    void powerOff(String name) {
        System.out.println(name + "가 전원을 끕니다.");
    }

    // 반환타입: O, 매개변수: O
    int plus(int x, int y) {
        int result = x + y;
        return result;
    }
}
```

8-5. 메서드 선언 및 호출

➢ 메서드 호출

- 메서드 호출은 메서드 블록을 실행하는 것이다.
- 객체 내부에서는 메서드 명으로 호출하고, 외부 객체에서는 참조 변수와 도트(.) 연산자로 호출한다.

```java
public class Calculator {
    //생성자
    public Calculator() {
        powerOn();          // 내부 호출
    }

    void powerOn() {
        System.out.println("전원을 켭니다.");
    }

    int plus(int x, int y) {
        int result = x + y;
        return result;
    }
}
```

```java
public class CalculatorEx {
    public static void main(String[] args) {
        Calculator calc = new Calculator();
        int result = calc.plus(100, 55);
        System.out.println("result: " + result);
    }
}
```

8-5. 메서드 선언 및 호출

- **return 문**
  - return 문은 메서드 실행을 종료하고 호출한 곳으로 돌아간다는 의미

- 일반적인 리턴(return) 사용

```
public class CalculatorEx {
    public static void main(String[] args) {
        int result = plus(100, 55);
        System.out.println("result: " + result);
    }

    private static int plus(int x, int y) {
        int sum = x + y;
        return sum;
    }
}
```

8-5. 메서드 선언 및 호출

➢ 리턴(return) 값을 그대로 사용

```
public class CalculatorEx {
    public static void main(String[] args) {
        System.out.println("결과: " + plus(plus(100, 50), plus(20, 30)));
    }

    private static int plus(int x, int y) {
        int sum = x + y;
        return sum;
    }
}
```

➢ 잘못된 리턴(return) 문 사용

```
public static int ex() {
    return 9;
    int x = 5;
}
```

return문 이후 실행문 실행 안됨
(에러: Unreachable Code)

8-5. 메서드 선언 및 호출

➢ **메서드 오버로딩(overloading)**

- 메서드 명은 동일하고 매개변수의 타입, 개수, 순서가 다른 메서드를 여러 개 선언하는 것이다.

```
public class CalculatorEx {
    public static void main(String[] args) {
        // 정사각형 넓이 구하는 메서드 호출
        System.out.println("정사각형 넓이: " + areaRectangle(100));
        // 직사각형 넓이 구하는 메서드 호출
        System.out.println("직사각형 넓이: " + areaRectangle(100, 50));
    }

    // 정사각형 넓이
    private static double areaRectangle(double width) {
        return width * width;
    }

    // 직사각형 넓이
    private static double areaRectangle(double width, double height) {
        return width * height;
    }
}
```

8-6. 정적 멤버

- static 키워드가 붙는 정적 멤버(필드와 메서드)는 객체 생성 없이 클래스를 통해 바로 사용 가능하다.

- **정적 멤버 선언**

```
public class Calculator {
    // 정적 필드 선언
    static double PI = 3.14159;

    // 정적 메서드 선언
    static int plus(int x, int y) {
        return x + y;
    }

    static int minus(int x, int y) {
        return x - y;
    }
}
```

8-6. 정적 멤버

➢ 정적 메서드 안에서 정적 멤버(필드와 메서드)만 가능

```
public class Calculator {
    String strCalc = "원의 면적: ";

    // 정적 필드 선언
    static double PI = 3.14159;

    static double calc(double radius) {
        double area = radius * radius * PI;
        System.out.println(this.strCalc + area);

        return area;
    }
}
```

❖ this 및 인스턴스 멤버 사용 불가

8-6. final 필드와 상수

➢ **final 필드**

- final 필드는 초기값이 저장되면 이 값이 최종 값이 되어 실행 도중에 수정이 불가하다.
- final 필드에서 초기값을 줄 수 있는 방법은 두가지가 있다.
  1. 필드 선언 시 초기값 대입
  2. 생성자에서 초기값 대입

```
public class Korean {
    final String name = "대한민국";
    final String ssn;

    public Korean(String ssn) {
        this.ssn = ssn;
    }
}
```

8-6. final 필드와 상수

➢ 상수

- 불변하는 값을 저장하는 필드를 상수(constant)라고 한다.
- 상수는 객체마다 저장할 필요가 없고, 여러 개의 값을 가져서도 안되기 때문에 static이면서 final 특성을 가져야 한다.

```
public class Earth {
    // 상수 선언 및 초기화
    static final double EARTH_RADIUS = 6400;

    // 상수 선언
    static final double EARTH_SURFACE_AREA;

    // 정적 블록에서 상수 초기화
    static {
        EARTH_SURFACE_AREA=4 * Math.PI * EARTH_RADIUS * EARTH_RADIUS;
    }
}
```

8-7. Getter와 Setter

- 객체의 필드(데이터)를 외부에서 마음대로 읽고 변경할 경우 객체의 무결성(결점이 없는 성질)이 깨질 수 있다. 그래서 객체 지향 프로그래밍에서는 직접적인 외부에서의 필드 접근을 막고 메서드를 통해 필드를 접근하는 것을 선호한다. 이런 역할을 하는 메서드가 Getter와 Setter이다.
  - Getter : 외부에서 객체의 필드를 읽을 때 필드 값이 외부 사용에 부적절할 경우 메서드로 적절한 값으로 변환한 후 리턴이 필요한 경우
  - Setter : 필드에 대한 외부 접근을 막고 메서드를 통해 데이터를 검증, 유효한 값만 필드에 저장이 필요할 경우

```
private String name;

public void setName(String name) {
    this.name = name;
}

public String getName() {
    return name;
}
```

필드 접근 제한자

메서드 명: set+필드 이름(첫 글자 대문자)
매개변수타입: 필드 타입

리턴 타입: 필드 타입
메서드 명: get+필드 이름(첫 글자 대문자)
리턴 값: 필드 값

8-7. Getter와 Setter

```
private String name;
private int speed;
private boolean stop;

public String getName() {
    return name;
}
public void setName(String name) {
    this.name = name;
}
public int getSpeed() {
    return speed;
}
public void setSpeed(int speed) {
    if(speed < 0) {
        this.speed = 0;
        return;
    } else
        this.speed = speed;
}
public boolean isStop() {
    return stop;
}
public void setStop(boolean stop) {
    this.stop = stop;
}
```

boolean 타입 is로 시작(관례)
메서드 명: is+필드 이름(첫 글자 대문자)
리턴 값: 필드 값

➢ 요점 정리

클래스 타입의 참조

- 클래스 타입 변수에는 객체 정보가 담겨 있는 힙 메모리의 번지가 들어 있다.
- 클래스 타입의 변수를 다른 변수에 대입하면 번지 정보가 복사 된다.
- 클래스 타입은 필드나 메서드의 매개 변수나 반환 타입으로 이용된다.

생성자

- 클래스명과 동일 명칭으로 반환 타입이 명시되지 않은 메서드가 생성자이다.
- new 연산자에 의한 객체 생성 후 자동실행 된다.
- 매개 변수가 있는 생성자는 new 연산자로 객체 생성 시 매개변수를 지정하면 해당 생성자를 실행할 수 있다.
- 클래스에 생성자 정의가 없으면 컴파일러는 기본 생성자를 추가해 준다.
- this는 객체 자신이며, this()를 사용하면 동일 클래스의 다른 생성자를 호출할 수 있다.

➢ 요점 정리

인스턴스와 클래스

- 인스턴스와 클래스는 완전히 다른 것이다.
- 클래스는 객체(인스턴스)를 생성하기 위한 설계도와 같은 것이다.
- 객체는 클래스(설계도)를 바탕으로 실제로 생성된 것을 말한다.

필드와 메서드

- 필드는 객체의 멤버로 데이터를 저장 처리한다.
- 메서드는 객체의 동작을 처리한다.

클래스 타입

- 클래스를 정의하면, 그 클래스 타입의 변수를 선언 할 수 있다.

인스턴스화

- new 연산자를 사용해서 클래스로부터 인스턴스를 생성
- 객체 생성 후 클래스 타입 변수에 도트(.)을 붙여 필드 및 메서드를 이용할 수 있다.

➢ 요점 정리

정적 멤버

- static 키워드가 붙는 정적 멤버(필드와 메서드)는 객체 생성 없이 클래스를 통해 바로 사용 가능하다.
- 정적 메서드는 내부에 정적 멤버(필드와 메서드)만 사용 가능하다.

final 필드와 상수

- final 필드와 상수는 초기값이 저장되면 값을 변경할 수 없다.
- final 필드는 필드 선언과 생성자에서 초기값을 줄 수 있다.
- 상수는 static이면서 final 특성을 갖는다.
- 상수는 필드 선언과 정적 블록에서 초기화될 수 있다.

➢ 예제 1

객체와 클래스에 대한 설명으로 틀린 것은 ?

① 클래스는 객체를 생성하기 위한 설계도(청사진)와 같은 것이다.

② new 연산자로 클래스의 생성자를 호출함으로써 객체가 생성된다.

③ 하나의 클래스로 하나의 객체만 생성할 수 있다.

④ 객체는 클래스의 인스턴스이다.

➢ 예제 2

클래스의 구성 멤버가 아닌 것은 ?

① 필드(field)

② 생성자(constructor)

③ 메서드(method)

④ 로컬 변수(local variable)

➢ 예제 3

필드, 생성자, 메서드에 대한 설명으로 틀린 것은 ?

① 필드는 객체의 데이터를 저장한다.

② 생성자는 객체의 초기화를 담당한다.

③ 메서드는 객체의 동작 부분으로, 실행 코드를 가지고 있는 블록이다.

④ 클래스는 반드시 필드와 메서드를 가져야 한다.

➢ 예제 4

생성자에 대한 설명으로 틀린 것은 ?

① 객체를 생성하려면 생성자 호출이 반드시 필요한 것은 아니다.

② 생성자는 다른 생성자를 호출하기 위해 this()를 사용할 수 있다.

③ 생성자가 선언되지 않으면 컴파일러가 기본 생성자를 추가한다.

④ 외부에서 객체를 생성할 수 없도록 생성자에 private 접근 제한자를 붙일 수 있다.

➢ 예제 5

메서드 오버로딩에 대한 설명으로 틀린 것은 ?

① 동일한 이름의 메서드를 여러 개 선언하는 것이다.

② 반드시 리턴 타입이 달라야 한다.

③ 매개변수의 타입, 수, 순서를 다르게 선언해야 한다.

④ 매개 값의 타입 및 수에 따라 호출될 메서드가 선택된다.

➢ 예제 6

인스턴스 멤버와 정적 멤버에 대한 설명으로 틀린 것은 ?

① 정적 멤버는 static으로 선언된 필드와 메서드를 말한다.

② 인스턴스 필드는 생성자 및 정적 블록에서 초기화될 수 있다.

③ 정적 필드와 정적 메서드는 객체 생성 없이 클래스를 통해 접근할 수 있다.

④ 인스턴스 필드와 메서드는 객체를 생성하고 사용해야 한다.

➢ 예제 7

현실 세계의 회원을 Member 클래스로 모델링하려고 합니다. 회원 데이터는 이름, 아이디, 패스워드, 나이, 전화번호, 주소가 있습니다.

이 데이터를 가지는 Member 클래스를 선언하세요.

데이터 이름	필드 이름	타입
이름	name	문자열
아이디	id	문자열
패스워드	password	문자열
나이	age	정수
전화번호	telno	문자열
주소	address	문자열

➢ 예제 8

예제 1에서 작성한 Member 클래스에 생성자를 추가하려고 합니다. 다음과 같이 name 필드와 id를 외부에서 받은 값으로 초기화 하도록 생성자를 선언해보세요.

```
Member user1 = new Member("홍길동", "hong");
```

➢ 예제 9

login() 메서드를 호출할 때에는 매개 값으로 id와 password를 제공하고, logout() 메서드는 id만 매개 값으로 제공하려고 합니다. 다음 조건을 보고 MemberService 클래스에서 login(), logout() 처리를 하는 메서드를 선언해보세요.

① login() 메서드는 매개 값이 id가 "hong", password가 "12345" 일 경우이만 true로 리턴

② logout() 메서드는 id + "님이 로그아웃 되었습니다." 가 출력되도록 할 것.

리턴 타입	메서드 이름	매개변수(타입)
boolean	login	id(String), password(String)
void	logout	id(String)

➢ 예제 10

println() 메서드는 매개 값을 콘솔에 출력합니다. println() 메서드의 매개 값으로 는 int, Boolean, double, String 타입의 값을 줄 수 있습니다. 다음 조건과 예제 코드를 보고 Printer 클래스에서 println() 메서드를 오버로딩하여 선언해보세요.

리턴 타입	메서드 이름	매개변수(타입)
void	println	value

```
Printer printer = new Printer();
printer.println(10);
printer.println(true);
printer.println(5.7);
printer.println("홍길동");
```